



ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Chương 2 **JAVA CORE**

Giảng viên: ThS. Nguyễn Thành Luân



Mục tiêu bài học

- ❖ Định danh
- ❖ Biến và hằng số
- ❖ Kiểu dữ liệu
- ❖ Toán tử & Biểu thức
- ❖ Cấu trúc điều khiển
- ❖ Lớp bao bọc
- ❖ Các ví dụ

Mục tiêu bài học

- ❖ Định danh
- ❖ Biến và hằng số
- ❖ Kiểu dữ liệu
- ❖ **Toán tử & Biểu thức**
- ❖ **Cấu trúc điều khiển**
- ❖ **Lớp bao bọc**
- ❖ Các ví dụ

Toán tử và Biểu thức (tt)

❖ Toán tử điều kiện

Cú pháp: <dieu_kien> ? <bieu_thuc_1> : <bieu_thuc_2>

Ví dụ:

int x = 10;

int y = 20;

int Z = (x < y) ? 30 : 40;

// Kết quả: z = 30 vì (x < y) đúng.

Toán tử và Biểu thức (tt)

```
int num;    // holds value of integer

// Create a Scanner object for keyboard input.
Scanner console = new Scanner(System.in);

// Get an integer.
System.out.print("Enter integer : ");
num = console.nextInt();

// Get absolute value of num
num = (num < 0) ? -num : num;

System.out.println("Absolute value is " + num);
```

Input: 5
Output:?

Input:-15
Output:?

Các cấu trúc điều khiển

- *if ... else*

Dạng 1:

```
if (<dieu_kien>) {  
    <cac_cau_lenh>;  
}
```

Dạng 2:

```
if (<dieu_kien>) {  
    <khoi_lenh_1>;  
}  
else {  
    <khoi_lenh_2>;  
}
```

Các cấu trúc điều khiển (tt)

```
int number;

// Create a Scanner object to read input.
Scanner console = new Scanner(System.in);

// Get number from the user.
System.out.print("Enter an integer: ");
number = console.nextInt();

// Determine even or odd.
if (number % 2 == 0)
{
    System.out.println("number is even");
}
else
{
    System.out.println("number is odd");
}
```

Các cấu trúc điều khiển (tt)

- *switch ... case*

```
switch (<bieu_thuc>) {  
    case <gia_tri_1>:  
        <khoi_lenh_1>;  
        break;  
  
    ....  
    case <gia_tri_n>:  
        <khoi_lenh_n>;  
        break;  
    default:  
        <khoi_lenh_mac_dinh>;  
}
```


Các cấu trúc điều khiển (tt)

Viết chương trình cho phép người dùng xếp loại học sinh dựa theo cấp độ mà người dùng nhập vào. Chương trình sẽ hiển thị ý nghĩa tương ứng của các cấp độ theo bảng sau:

Cấp độ	Ý nghĩa
A	Xuất sắc
B	Tốt
C	Trung bình
D	Yếu
F	Kém

Các cấu trúc điều khiển (tt)

```
char grade; // To hold grade

// Create a Scanner object to read input.
Scanner console = new Scanner(System.in);

// Get grade from the user.
System.out.print("Enter grade: ");
grade = console.next().charAt(0);

// Determine and display grade
switch (grade)
{
case 'A':
    System.out.println("Excellent");
    break;
case 'B':
    System.out.println("Good");
    break;
case 'C':
    System.out.println("Average");
    break;
case 'D':
    System.out.println("Deficient");
    break;
case 'F':
    System.out.println("Failing");
    break;
default:
    System.out.println("Invalid input");
}
```

Vòng lặp điều khiển (Loop Controls)

❖ Dạng 1:

```
while (<dieu_kien>) {  
    <khoi_lenh>;  
}
```

❖ Dạng 2:

```
do {  
    <khoi_lenh >;  
} while (dieu_kien);
```

❖ Dạng 3:

```
for (statement 1; statement 2; statement 3) {  
    <khoi_lenh >;  
}
```

Vòng lặp điều khiển (tt)

```
int count = 1;    // count is initiliazed

while (count <= 10)    // count is tested
{
    System.out.print(count + " ");
    count++;           // count is changed
}
```

Vòng lặp điều khiển (tt)

```
int value;        // to hold data entered by the user
int sum = 0;       // initialize the sum
char choice;      // to hold 'y' or 'n'

// Create a Scanner object for keyboard input.
Scanner console = new Scanner(System.in);

do
{
    // Get the value from the user.
    System.out.print("Enter integer: ");
    value = console.nextInt();

    // add value to sum
    sum = sum + value;

    // Get the choice from the user to add more number
    System.out.print("Enter Y for yes or N for no: ");
    choice = console.next().charAt(0);
}
while ((choice == 'y') || (choice == 'Y'));
```

Vòng lặp điều khiển (tt)

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String i : cars) {  
    System.out.println(i);  
}
```

Lệnh Break

- ❖ Dùng để thoát ra khỏi một vòng lặp hoặc switch.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    System.out.println(i);  
}
```

Lệnh Continue

- ❖ Tiếp tục với lần lặp tiếp theo trong vòng lặp.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```


Đầu vào từ CONSOLE

❖ Lớp java.util.scanner

public boolean	nextBoolean()
public byte	nextByte()
public byte	nextByte(int radix)
public double	nextDouble()
public float	nextFloat()

public int	nextInt()
public <u>String</u>	nextLine()
public long	nextLong()

Lớp bao bọc (Wrapper Class)

Kiểu dữ liệu	Lớp Wrapper (java.lang.*)	Ghi chú
boolean	Boolean	- Gói: có thể chứa hai hoặc nhiều lớp. - Bên cạnh các Lớp Wrapper, gói java.lang cung cấp các lớp cơ bản để hỗ trợ các thiết kế trong Java: String, Math, ...
byte	Byte	
short	Short	
char	Character	
int	Integer	
long	Long	
float	Float	
double	Double	

Hướng dẫn bài tập Chương 2

Bài tập

Bài 1: Viết chương trình Java cho phép người dùng nhập vào một số nguyên dương n , thực hiện các công việc sau:

- a. Tìm chữ số đầu tiên của n .
- b. Kiểm tra n có toàn chữ số lẻ hay không.
- c. Tính tích tất cả các “ước số” của n .
- d. Kiểm tra n có phải số hoàn thiện hay không.

Bài tập

Bài 2: Viết chương trình Java cho phép người dùng nhập vào một chuỗi, thực hiện các yêu cầu sau:

- a. Đếm số lượng của các chữ cái, khoảng trắng, số và các ký tự khác (ngoài những cái trước đó) trong chuỗi.

Ví dụ:

Input: “Aa kiu, I swd skieo 236587. GH kiu: sieo?? 25.33”

Output: Chữ cái: 23, Khoảng trắng: 9

Số: 10, Các ký tự khác: 6

- b. Tìm ký tự có số lần xuất hiện nhiều thứ hai trong chuỗi (Nếu có nhiều ký tự có cùng số lần xuất hiện nhiều thứ hai, in ra ký tự đầu tiên).
- c. Tính tổng các số xuất hiện trong chuỗi.

Ví dụ:

Input: “It 15 is25 a 20string”

Output: Tổng các số trong chuỗi: 60

Bài tập

Bài 3: Thực hiện các yêu cầu sau:

Viết chương trình Java để kiểm tra xem có tồn tại mảng con được tạo thành bởi **các số nguyên liên tiếp** từ một mảng số nguyên cho trước hay không. Nếu có in ra mảng con có độ dài lớn nhất, nếu không in ra Không tồn tại.

Ví dụ:

Input: [22, 5, 0, 2, 1, 4, 3, 6, 1, 23, 8]

Các mảng con được tạo thành từ các số nguyên liên tiếp là:

[22, 23] và [0, 1, 2, 3, 4, 5, 6]

Output: [0, 1, 2, 3, 4, 5, 6]

Bài 4: Viết chương trình Java để đếm **số tam giác có thể có** từ một mảng các số nguyên dương chưa được sắp xếp cho trước.

Gợi ý: Sử dụng tính chất của bất đẳng thức tam giác: “Tổng độ dài của hai cạnh bất kỳ của tam giác phải lớn hơn hoặc bằng độ dài của cạnh thứ ba”.

Ví dụ:

Input: [6, 7, 9, 16, 25, 12, 30, 40]

Output: Số tam giác có thể có: 17

Bài tập

Bài 5 (*): Viết chương trình Java cho phép người dùng nhập vào một ma trận số thực M có kích thước $m \times n$. Thực hiện các công việc sau:

- Tìm phần tử chẵn dương nhỏ nhất trong ma trận.
- Tính giá trị trung bình của các phần tử nhỏ nhất trên mỗi cột của một ma trận.
- Xóa dòng có tổng lớn nhất trong ma trận.
- Kiểm tra xem M có phải là ma trận vuông không? Nếu có, kiểm tra tiếp xem M có là ma trận tam giác trên hay ma trận tam giác dưới không?
- Tạo thêm một ma trận mới N có kích thước là $n \times m$ (m và n là kích thước của ma trận M ban đầu). Tính tích hai ma trận M và N .
- Chuyển vị ma trận tích tính được từ ý trên.

Hỏi & Đáp

Cảm ơn!